

Report Lab 2

Computer Network Configuration

RC - L.EIC025 | 1S 2025
Class 08

André Gomes, up202304252
André Pinho, up202307008

Summary

This laboratory work focused on the practical implementation of a complete network infrastructure, ranging from Layer 1 (Physical) to Layer 7 (Application) of the TCP/IP model. The objective was to develop a C-based FTP client and configure a network topology capable of supporting it. The work was divided into two parts: software development and network configuration. In Part 1, we implemented a functional FTP client capable of parsing URLs, resolving hostnames via DNS, and transferring files using the FTP protocol (Active/Passive modes). In Part 2, we built the network infrastructure step-by-step. We started with a single broadcast domain, introduced network segmentation using bridges (VLAN simulation), and implemented routing using both a Linux-based router and a commercial MikroTik router. We further enabled Internet connectivity through Network Address Translation (NAT) and configured DNS. Throughout the experiments, Wireshark was used to capture and analyze traffic, allowing us to verify protocols such as ARP, ICMP, and TCP, and to visualize network congestion and flow control mechanisms.

Introduction

This report intends to present how a simple FTP client was implemented and how a network was configured in order to use said client. The application layer development covers Client-Server concepts as well as the FTP, a protocol which is a part of the TCP/IP suite. The network configuration provides understanding of switching/bridging, IP addresses, routing, ARP, NAT and DNS.

Part 1 - Download application

Architecture

The application, developed in C, consists of two header files, `getip.h` and `clientTCP.h`, and the corresponding functions implementation in `getip.c` and `clientTCP.c`, a core file, `app.c`, a `makefile` and a `README.md`.

- `getip.h/c`: ip resolution via the hostname
- `clientTCP.h/c` TCP communication (connecting, sending commands and receiving replies).
- `app.c`: URL parsing, program flow, feedback.
- `Makefile`: ease of compilation, proper clean up.
- `README.md`: Project description, features, usage examples, ...

FTP client flow:

- Parse URL -> Resolve IP -> Connect -> Login -> Passive Mode -> Connect -> Download -> Quit

The URL parsing consists of splitting it into user, pass, host and path. This is done by a call to the function `parse ftp_url`.

Given that there are three login options, username and password, username only and anonymous, our code is capable of detecting what is provided and returns the appropriate content, it also displays error messages in case of an improper URL.

IP resolution is handled in the `getip.c` module via the `resolve_hostname_to_ip` function. This function utilizes the standard DNS resolver through the `gethostbyname()` system call.

When a hostname (e.g., `ftp.netlab.fe.up.pt`) is provided, `gethostbyname()` returns a pointer to a `hostent` structure. This structure contains the official name of the host and a list of its network addresses (`h_addr_list`). The application extracts the first address from this list (`h_addr_list[0]`) and converts it from binary network byte order into a human-readable IPv4

dotted-decimal string (e.g., 192.168.1.1) using `inet_ntoa()`. This string is then used by the TCP module to establish the control connection."

The connection is made by calling the function `tcp_connect` with the resolved ip address and the standard FTP port (21), the result is stored in a socket file descriptor which acts as a handle for the active connection and is used to validate the connection and fetch the appropriate response using `tcp_receive_response`.

For the login, the user command is first sent via `tcp_send_command`, using the established socket field descriptor, afterwards the server response is fetched and analysed, this determines whether the the username is all the server requires or if the password must be provided, in the later the password is sent in the same manner the user is and the response is once again analysed for success or failure.

Upon successful login the client enters passive mode, once again `tcp_send_command` is used to send the command and the response is fetched. This response contains the ip and port for the data connection in the format (ip1,ip2,ip3,ip4,p1,p2), where ip1 to ip4 represent parts of the ip address in order, p1 and p2 allow to calculate the port, $\text{port} = (\text{p1} \times 256) + \text{p2}$. The decoding and calculation is performed by the `parse_pasv_response` function which constructs the ip address, stores it in a variable and returns the port.

A new socket file descriptor is created using the ip address and port attained in the last phase, which handles the actual file transfer independently of the command channel.

The RETR command, with the file path is then sent over the control connection to instruct the server to begin transmitting the file

Afterwards the file name is extracted from the user input using `extract_filename`, a local file is created in binary mode to store the downloaded content and various control variables are set, the data is read from the data socket in a loop using `recv` and written to the file. The loop ends when `recv` returns a number smaller than 1, indicating the transfer is complete.

Finally, the quit command is used to close the connection.

For security purposes, all user dependent text field lengths are validated, they all have a maximum size and all copies account for said size. Moreover, all socket file descriptors are closed in case of error.

In order for the user to be aware of the program state, all phases output their result, success and description or error, all tcp responses are printed fully and when downloading, there is an updating percentage and byte count.

The program can be compiled using the Makefile by running **make** in the root directory, this generates an executable named **download**.

The program now can be executed by running **./download <ftp_url>**.

Example

In order to prove that the application is working, a small video will be downloaded from ftp.gnu.org, providing only a username.

By running **./download ftp://anonymous@ftp.gnu.org/video/FSF_event_small.ogv** in the root directory after compilation, the video will be downloaded.

The screenshots in the annexes show the terminal window where the application was compiled and ran and the wireshark log.

Terminal Analysis: The terminal screenshot validates that the ftp_url was properly parsed, shows every server response and that all interactions were successful and the port calculation is correct, as well as the file size extraction.

Wireshark Analysis: "The Wireshark logs captured during the execution validate the protocol implementation:

1. **Control Connection & Login:** The logs show the initial TCP 3-way handshake on port 21. Packet 7 shows the client USER anonymous request, followed by the server's confirmation 230 Login successful in Packet 9.
2. **Passive Mode:** Packet 13 shows the PASV command. The server responds in Packet 15 with code 227 Entering Passive Mode. The parameters (79, 253) correspond to the port $79 \times 256 + 253 = 20477$, which matches the port used for the data connection.
3. **Data Request:** After establishing the secondary TCP connection (Packet 17), the client creates the RETR ... request in Packet 20. The server accepts this with code 150 Opening BINARY mode... in Packet 23.
4. **Transfer & Termination:** The actual file content is visible as FTP-DATA packets. Once the transfer finishes, the server sends 226 Transfer complete (Packet 1622), and the client closes the session with QUIT (Packet 1625), receiving a final 221 Goodbye (Packet 1627)."

[Terminal screenshot](#) [Wireshark logs](#)

Part 2 - Network configuration and analysis

This section describes the progressive implementation of a network topology, starting from a single broadcast domain up to a routed network with NAT and DNS, analyzing the protocols involved at each stage. We will explain, for each experiment, its network architecture, objectives, main configuration commands and their use, and finally the relevant logs of the experiment.

Experiment 1: Configure an IP Network

The objective of this experiment was to configure a simple IP network within a single broadcast domain. Two Linux hosts (tux123 and tux124) were connected to a MikroTik switch acting in its default configuration (all ports switched). The goal was to establish connectivity using IPv4 and analyze the Address Resolution Protocol (ARP).

To do so, we first needed to connect the cables from if_e1 of tux123 and tux124 to the switch (ports 1 and 24 respectively in our case) and after configure if_e1 interface of tux123 and if_e1 interface of tux124 ([Configuring tux123 and tux124](#)).

Following this we verified the connectivity between the computers by using **ping 172.16.120.254** and then we saved the IP and MAC addresses of the network interfaces after running **arp -n** in tux123 (IP - 172.16.120.254; MAC - ec:75:0c:c2:3c:ac). In order to observe the ARP mechanism, the ARP table entries were deleted in tux123 (**arp -d 172.16.120.254**) before initiating the ping. We started the Wireshark capture and using **ping 172.16.120.254** we gathered the following log: [Log experiencia 1](#)

The log reveals the following sequence of patterns:

1. ARP Request: tux123 broadcasts "Who has 172.16.120.254? Tell 172.16.120.1".
2. ARP Reply: tux124 replies unicast to tux123 with its MAC address.
3. ICMP Echo Request/Reply: Only after resolving the MAC address does the first ICMP Echo Request leave tux123.

Questions:

ARP (Address Resolution Protocol) packets map known IP addresses (Layer 3) to unknown MAC addresses (Layer 2). This is necessary because Ethernet switches and network cards communicate using physical MAC addresses, not IP addresses.

ARP Requests use the sender's MAC/IP as the source and the Broadcast MAC (FF:FF:FF:FF:FF:FF) as the destination because the target's location is unknown. ARP Replies use the target's MAC/IP as the source and the requester's MAC as the destination (unicast) because the requester's identity is now known.

The ping command generates ICMP Echo Request packets and waits for ICMP Echo Reply packets to confirm connectivity.

Ping packets use the specific unicast MAC and IP addresses of the source and destination devices for both Requests and Replies, as the address resolution (ARP) must have already completed successfully for the ping to be sent.

A receiving frame is identified by the EtherType field in the Ethernet header: 0x0806 indicates ARP, and 0x0800 indicates IPv4. If it is IPv4, the "Protocol" field in the IP header identifies the payload, where 1 indicates ICMP.

The frame length is determined physically by the hardware detecting the end of the signal, or logically by reading the "Total Length" field in the IP header and adding the Ethernet overhead (14-byte header + 4-byte FCS).

The loopback interface (lo, usually 127.0.0.1) is a virtual network interface that allows a computer to communicate with itself. It is essential for internal system communication and for testing network applications without needing physical network hardware.

Experiment 2: Implement two bridges in a switch

This experiment was designed in order to make us verify that different networks do not communicate between each other (Unicast Isolation) and to verify how far broadcast packets travel (Broadcast Isolation). To do so, the switch was configured to create two logically separated networks (Broadcast Domains) using Bridges, effectively simulating VLANs.

- Bridge120 (172.16.120.0/24): Ports 1 (tux123 - if_e1) and 24 (tux124 - if_e1).
- Bridge121 (172.16.121.0/24): Port 16 (tux122 - if_e1).

To do this, we needed to make sure the previous computers kept the same configuration and then configured the IP of the new computer ([Configuring IP](#)) and saved the IP and MAC addresses (IP - ; MAC -). After that, we configured the switch by creating two virtual bridges to separate the traffic ([GTKTerm Configuring Switch](#)). At this point, we removed the default bridges from the ports 1, 24 and 16 of the switch, in order for these to be able to be added to the new bridges ([Adding Bridges](#)).

After this, we tested the Unicast connectivity to check if the networks are isolated ([Pings Exp2](#)). The Wireshark capture from these pings verified a successful first ping ([Ping from tux123 to tux124 and tux122](#)) because they are in the same bridge. Contrary to this, the second ping failed because they are on different bridges.

The second part of this experiment was the broadcast test and to do so, we recorded using Wireshark in all tux's and did **ping -b 172.16.120.255** from tux123 ([Broadcast from 3](#)) to do a broadcast ping in the network 120. The -b flag enables broadcast and the IP ending in .255 is the broadcast address of this subnet. The capture on tux124 ([Broadcast 3 to 4](#)) shows the incoming broadcast packets, as it shares bridge120. However, the capture on tux122 ([Broadcast 3 to 2](#)) shows no packets from this source. We repeated these steps and in the tux122 we did **ping -b 5 172.16.121.255** ([Broadcast from 2](#)) and the results showed

that neither tux123 ([Broadcast 2 to 3](#)) or 124 ([Broadcast 2 to 4](#)) acknowledged this communication because tux122 is alone in bridge 121.

With this experiment, we arrived at the conclusion that the bridges successfully isolated the collision and broadcast domains at Layer 2. Tux123 could not communicate with tux122 because they were on different logical switches with no router to forward packets between them.

Questions:

To configure bridge120, we first create the logical bridge interface using the command `/interface bridge add name=bridge120`. Then, we assign the specific physical ports to this bridge using `/interface bridge port add bridge=bridge120 interface=etherX`, ensuring these ports were previously removed from any default configuration.

There are two distinct broadcast domains. We conclude this from the logs because broadcast packets sent to 172.16.120.255 were captured on tux124 (which shares bridge120) but were completely invisible on tux122 (which is on bridge121), proving that broadcast traffic is contained within the configured bridge and does not cross to the other.

Experiment 3: Configure a Router in Linux

The main goal of this experiment is to configure tux124 as a Router. To do so, it was physically connected to both bridges (if_e1 to bridge120 and if_e2 to bridge121) and then configured in the GTKTerm in tux122 ([exp3_b1](#)). After this we had to configure tux124 to forward traffic between the two interfaces ([exp3_b2](#)) and also reconfigure tux123 and tux122 so that each of them can reach the other using tux124 ([exp3_b3](#)). At this point we cleaned the ARP tables from all tux's (`arp -d -a`) and started recording using Wireshark in tux123 (if_e1) in order to see the results of the following ping's - [exp3_b4](#). Connectivity was verified in three stages from tux123:

- Local Gateway ([Ping 172.16.120.254](#)): The ping succeeded with ttl=64, confirming tux123 and the router's ingress interface share the same Layer 2 segment.
- Remote Gateway Interface ([Ping 172.16.121.253](#)): The successful ping demonstrates that tux123 can reach the router's interface, validating the routing path across the device.
- Remote Host ([Ping 172.16.121.1](#)): The ping reached tux122 successfully. Crucially, the return ttl=63 confirms the packet traversed exactly one router (tux124).

To analyze the routing process in detail, we proceeded to capture traffic directly on the router (tux124). Two instances of Wireshark were launched simultaneously: one monitoring if_e1 and the other monitoring if_e2. Before initiating traffic, the ARP tables were cleared on all devices to ensure the address resolution process was visible. A ping was then sent from tux123 to tux122. The results are displayed in [Ping tux122](#), where the left window shows the traffic on if_e1 and the right window shows the traffic on if_e2. The captures clearly illustrate the separation of broadcast domains and the routing mechanism:

ARP Resolution:

- On if_e1: tux123 broadcasts an ARP request for its gateway, 172.16.120.254. The router replies with the MAC address of its if_e1 interface.
- On if_e2: After receiving the IP packet destined for the 121 network, the router broadcasts an ARP request on if_e2 to find 172.16.121.1 (tux122). tux122 replies with its MAC address.

ICMP Forwarding:

- Echo Request: The packet from tux123 arrives at the router on if_e1. The router then forwards it out of if_e2 towards tux122.

- Echo Reply: The reply from tux122 is received by the router on if_e2 and is subsequently routed back to tux123 via if_e1.

This confirms that tux124 is correctly forwarding packets between the interfaces and managing ARP resolution independently for each subnet.

Questions:

The routing tables contain direct connection routes (e.g., 172.16.120.0/24) that allow communication with devices on the local link, and gateway routes (default or static) that direct traffic for remote networks to a specific router IP address.

A forwarding table entry contains the Destination Network (target subnet), the Gateway (next hop IP), the Genmask (subnet mask), the Flags (indicating route status), and the Interface (network card) used to transmit the packet.

We observe ARP Requests (broadcast) from the sender to resolve the router's MAC, followed by the router's ARP Reply. On the second interface, the router sends its own ARP Request to resolve the final destination's MAC, followed by the destination's Reply.

ICMP Echo Request and Echo Reply packets are observed to verify end-to-end connectivity. We may also see ICMP Time Exceeded if a loop occurs, or ICMP Redirect if the router detects a more efficient path.

The IP addresses (Source and Destination) remain constant throughout the entire path. The MAC addresses change at each hop: the sender uses the Router's MAC as the destination, and the Router replaces it with the final recipient's MAC for the next leg of the journey.

Experiment 4: Configure a Commercial Router and Implement NAT

In this experiment, we will introduce a commercial MikroTik router (RC) into the network to provide access to the external laboratory network (172.16.1.0/24) and the Internet. The Linux router (tux124) remained active. NAT (Network Address Translation) was implemented on RC to allow the private lab networks to communicate with the outside. Regarding physical connections, we had to connect ether1 of RC to the lab network and ether2 to port 10 of the Switch (bridge 121). We configured the Switch to add port 10 ([Configuring Switch](#)) and disconnected the cable from the Switch Console port and connected it to the Console port of the RC. After resetting it we configured the IP addresses on the RC by configuring the LAN interface (ether2) and the WAN interface (ether1) ([Configure LAN and WAN interfaces](#)). After this we configured the routes and NAT on the RC, because it needs to be able to identify how the "Internet" and network 120 are accessed ([Configure Routes and NAT on the Router](#)). Once the configuration was complete, we updated the routing tables ([Update Routes](#)). We verified connectivity by pinging from tux123 to all relevant interfaces ([Ping from 3 to server step7 fails](#); [Ping from 3 to Server step5](#)). The results confirmed that tux123 could successfully reach tux122, tux124, and the RC interfaces.

Next, we explored ICMP redirect behavior on tux122. We disabled ICMP redirect acceptance ([Disable ICMP Redirects](#)) and changed the routing to use RC as gateway instead of tux124 ([Change Gateway to RC](#)). When pinging tux123 from tux122, the Wireshark capture ([Ping from 2 to 3 via Router](#)) showed packets being forwarded through RC even though a more direct path existed. We performed a traceroute which confirmed this

routing path. We then restored the route through tux124 ([Restore Gateway to tux124](#)) and observed the more direct path ([Ping from 2 to 3 redirect step 4](#)). To understand ICMP redirects, we removed the route and re-enabled redirect acceptance ([Enable ICMP Redirects and Test](#)). When pinging, tux122 initially used the default gateway (RC), but ICMP redirect messages instructed it to use the optimal path through tux124. We then tested connectivity from tux123 to the external FTP server at 172.16.1.10. With NAT enabled, packets successfully traversed the network because RC performed source NAT ([Ping from 3 to server](#)). Finally, we disabled NAT ([Disable NAT](#)) and attempted the same ping, which failed because return packets could not reach our private network ([Ping from 3 to server after disabling NAT](#)). This demonstrated that NAT is essential for allowing private networks to communicate with external networks.

Questions:

A static route is configured using the command `/ip route add dst-address=[Destination_Network] gateway=[Next_Hop_IP]`, which manually defines the path traffic must take to reach a specific remote subnet.

Without ICMP Redirects, packets travel from the host to the Default Gateway and then back to the Router (hairpinning), which is inefficient. With Redirects enabled, the Gateway sends an ICMP Redirect message, telling the host to send subsequent packets directly to the Router, optimizing the path.

NAT is configured using the command `/ip firewall nat add chain=srcnat action=masquerade out-interface=[WAN_Interface]`, which enables the router to translate internal private IP addresses to its single public interface IP.

NAT (Network Address Translation) modifies the IP header of outgoing packets, replacing the private source IP address with the router's public IP address, allowing private devices to communicate with the Internet.

When NAT is disabled, the ping fails because the packets reach the external server with their original private source IP. The external server drops the reply because private IP ranges are not routable on the public Internet and it has no path back to the sender.

Experiment 5: DNS

Domain Name System (DNS) configuration was added to translate human-readable names into IP addresses. The resolvers were configured to use the Lab Gateway (172.16.120.254) as the nameserver. This ensures that DNS queries are sent through the experimental network interface (if_e1) and can resolve both internal lab domains and external internet domains via the router's DNS relay.

The configuration involved editing `/etc/resolv.conf` to set the nameserver to 172.16.120.254 and ensuring the default route was set to the gateway using `route add default gw 172.16.120.254 dev if_e1`.

Using the command `ping google.com` triggered a DNS resolution sequence. We observed an IPv4 UDP packet sent to destination port 53 on the gateway (172.16.120.254) asking for the IP of the hostname. A response packet was received containing the Answer Section with the resolved IP address. Connectivity was only established after this successful resolution, proving the dependency of application-layer services on correct DNS configuration. ([Ping google.com](#))

Questions:

A DNS packet is encapsulated in a UDP datagram (Layer 4), which is inside an IP packet (Layer 3), which is finally inside an Ethernet Frame (Layer 2).

The destination port for DNS queries is typically UDP 53.

The DNS server is running on the Lab Gateway (172.16.120.254), which acts as a DNS relay to forward queries to the appropriate internal or external DNS servers.

We observe a DNS Standard Query (A) packet asking for the IP of the hostname, followed by a Standard Query Response packet containing the "Answer" section with the resolved IP address.

The ARP protocol is still necessary because even after DNS resolves the hostname to an IP address, the computer needs the MAC address of the next-hop gateway to construct the Ethernet frame required to transmit the packet.

Experiment 6: TCP Connections

The experiment analyzed TCP protocol behavior focusing on connection establishment, data transfer, and congestion control using a custom FTP client. We first verified the application functionality by downloading a file from the lab server, confirming correct execution of FTP commands like USER, PASS, PASV, and RETR. ([step 2](#))

FTP utilizes two distinct TCP connections. The Control Connection handles authentication and commands, evidenced by server response code 227 entering passive mode. ([step 4 ftp filter](#))

The Data Connection establishes upon the RETR command. The TCP flow graph shows the standard 3-way handshake (SYN, SYN-ACK, ACK), followed by data transfer segments, and finally the connection termination via FIN-ACK. ([step4 tcp](#)) ([step 4 all flows](#))

We analyzed congestion control during a single file download. The Stevens graph shows a linear progression of sequence numbers, indicating stable throughput where the connection saturated available bandwidth without significant packet loss. ([step 4 congestion control](#))

The concurrent flow experiment revealed distinct starvation behavior. Tux3, the established flow, utilized the full 80 Mbps bandwidth immediately and maintained saturation until completion around 11.5 seconds. ([Sametime final tux3](#))

In contrast, Tux4 attempted to connect while Tux3 saturated the link. Tux4 suffered starvation with near-zero throughput for the first 11 seconds due to buffer saturation by Tux3. Once Tux3 finished, Tux4's throughput immediately spiked to capture the available bandwidth. ([Sametime final tux4](#))

The results validate TCP reliability mechanisms but highlight that without fair queuing, established flows can starve new connections.

Questions:

A TCP connection is established using a 3-way handshake. The client initiates the connection by sending a SYN packet, the server replies with a SYN-ACK packet to acknowledge and synchronize sequence numbers, and the client finally confirms with an ACK packet, establishing the session.

The connection is terminated using a 4-way handshake. The client (or server) sends a FIN packet to indicate it has no more data to send. The other side sends an ACK to acknowledge the request, then sends its own FIN when it is ready to close, which is finally acknowledged by the first party with an ACK.

The throughput graph shows distinct phases of TCP congestion control. It begins with the Slow Start phase, where throughput increases exponentially. When packet loss occurs (indicated by a sharp drop in the graph), the protocol reduces the transmission rate and enters Congestion Avoidance, where throughput increases linearly to probe for available bandwidth without causing immediate congestion.

Throughput is controlled by two main window parameters: the Flow Control Window (rwnd), which is advertised by the receiver to prevent buffer overflow, and the Congestion Window (cwnd), which is calculated by the sender to prevent network saturation. The actual transmission rate is determined by the minimum of these two windows.

Conclusions

This project successfully integrated software development with network engineering.

- **Application Layer:** We learned that protocols like FTP rely on strict state machines (Login -> Passive -> Transfer) and that the application layer is agnostic to the underlying network complexity, relying on DNS and IP routing to function.
- **Network Layer:** We verified that bridges isolate collision domains but not broadcast domains (unless configured as separate VLANs/bridges), whereas routers completely isolate broadcast domains.
- **NAT & Routing:** Experiment 4 demonstrated that without NAT, private networks are "invisible" to the outside world. The routing experiments highlighted the importance of correct static routes; a single missing return route prevents two-way communication (as seen in the "No route to host" errors).
- **Diagnostics:** Tools like ping, traceroute, and Wireshark were indispensable for diagnosing issues like incorrect ARP entries or routing loops.

By completing this lab, we gained a practical understanding of how data moves from a C program, through the socket API, encapsulated into TCP/IP/Ethernet packets, routed across physical devices, and delivered to a destination.

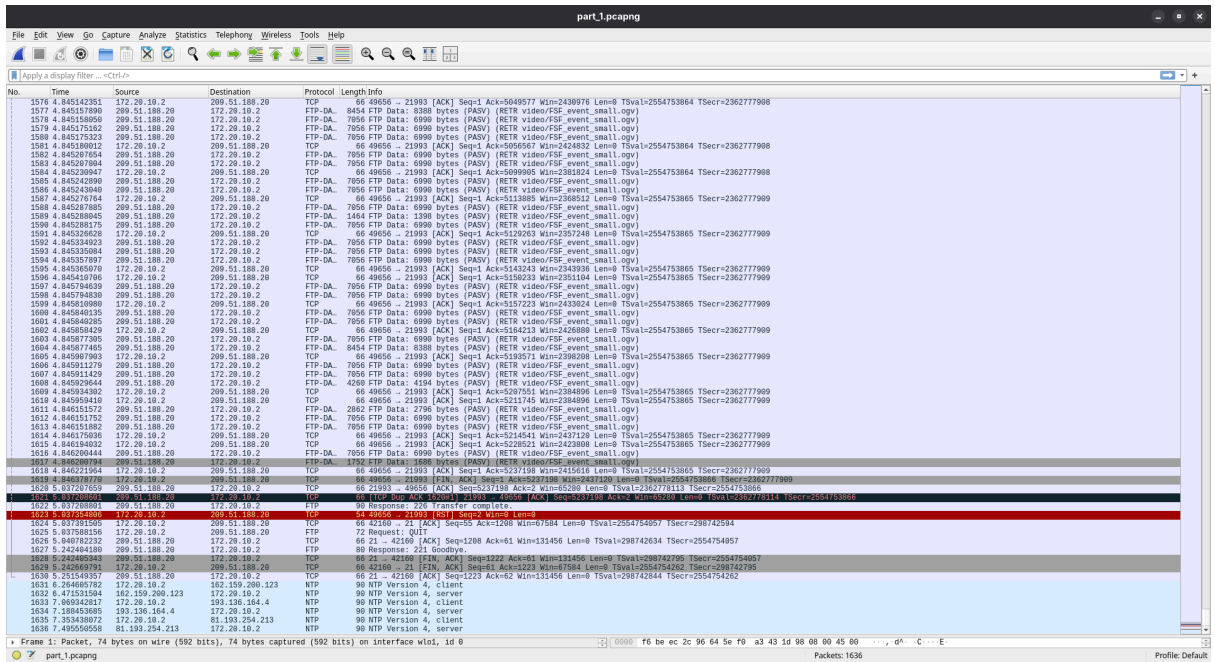
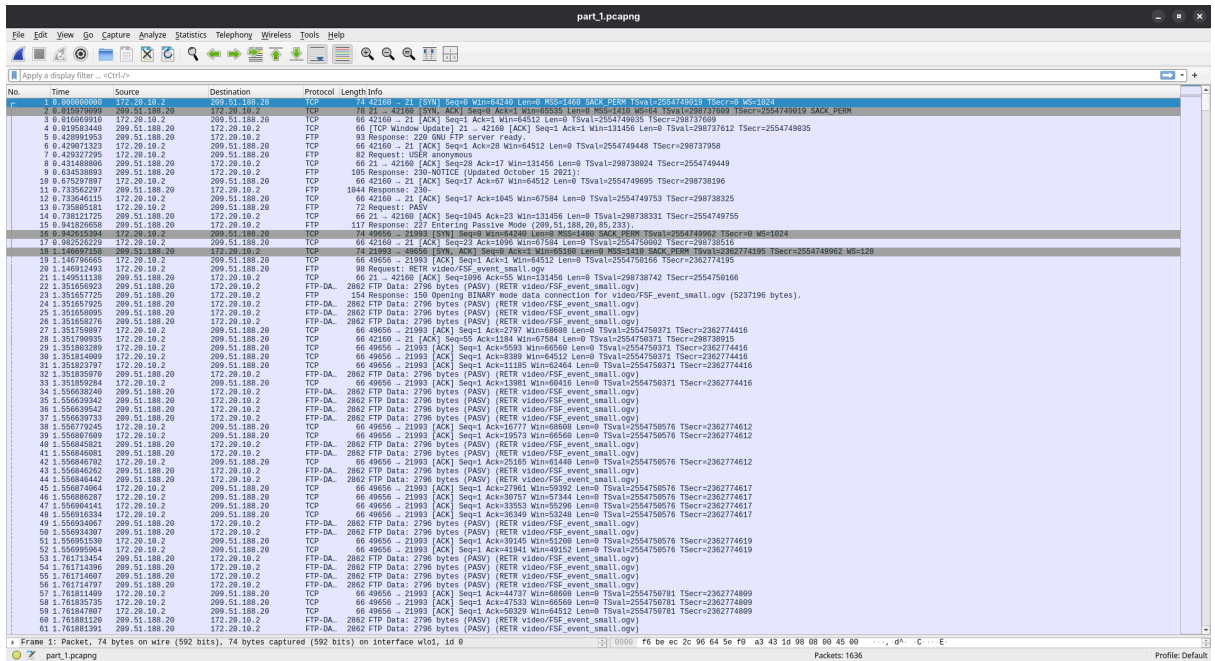
Annexes

Part 1

Terminal

```
➔ application git:(main) X ls
app.c clientTCP.c clientTCP.h getip.c getip.h Makefile README.md
➔ application git:(main) X make
gcc -Wall -Wextra -g -c app.c -o app.o
gcc -Wall -Wextra -g -c clientTCP.c -o clientTCP.o
gcc -Wall -Wextra -g -c getip.c -o getip.o
gcc -Wall -Wextra -g -o download app.o clientTCP.o getip.o
➔ application git:(main) X ./download ftp://anonymous@ftp.gnu.org/video/FSF_event_small.ogv
Connecting to ftp.gnu.org...
Resolved to IP: 209.51.188.20
Server: 220 GNU FTP server ready.
Server: 230-NOTICE (Updated October 15 2021):
230-
230-If you maintain scripts used to access ftp.gnu.org over FTP,
230-we strongly encourage you to change them to use HTTPS instead.
230-
230-Eventually we hope to shut down FTP protocol access, but plan
230-to give notice here and other places for several months ahead
230-of time.
230-
230-
230-
230-Due to U.S. Export Regulations, all cryptographic software on this
230-site is subject to the following legal notice:
230-
230-   This site includes publicly available encryption source code
230-   which, together with object code resulting from the compiling of
230-   publicly available source code, may be exported from the United
230-   States under License Exception "TSU" pursuant to 15 C.F.R. Section
230-   740.13(e).
230-
230-This legal notice applies to cryptographic software only. Please see
230-the Bureau of Industry and Security (www.bxa.doc.gov) for more
230-information about current U.S. regulations.
230 Login successful.
Logged in successfully.
Server: 227 Entering Passive Mode (209,51,188,20,79,253).
Data connection: 209.51.188.20:20477
Server: 150 Opening BINARY mode data connection for video/FSF_event_small.ogv (5237196 bytes).
File size: 5237196 bytes
Downloading file to: FSF_event_small.ogv
Downloaded: 100% (5237196 bytes total)
Server: 226 Transfer complete.
Download completed successfully!
Server: 221 Goodbye.
➔ application git:(main) X ls
app.c app.o clientTCP.c clientTCP.h clientTCP.o download FSF_event_small.ogv getip.c getip.h getip.o Makefile README.md
```


Wireshark



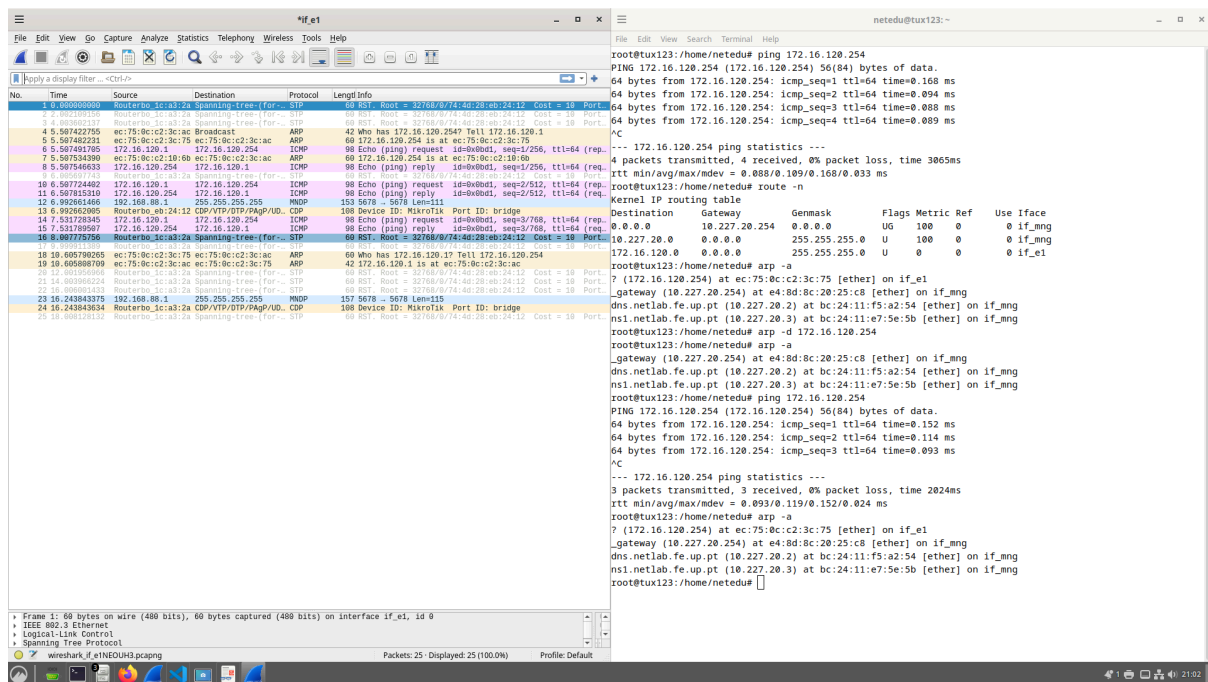
Part 2

Exp 1

Configuring tux123 and tux124

- ifconfig if_e1 172.16.120.1/24 up #from tux123
- ifconfig if_e1 172.16.120.254/24 up #from tux124

Log experiencia 1



Exp 2

Adding Bridges

- /interface bridge port remove [find interface=ether1]
- /interface bridge port remove [find interface=ether24]
- /interface bridge port remove [find interface=ether16]
- /interface bridge port add bridge=bridge120 interface=ether1
- /interface bridge port add bridge=bridge120 interface=ether24
- /interface bridge port add bridge=bridge121 interface=ether16

GTKTerm Configuring Switch

- /interface bridge add name=bridge120
- /interface bridge add name=bridge121

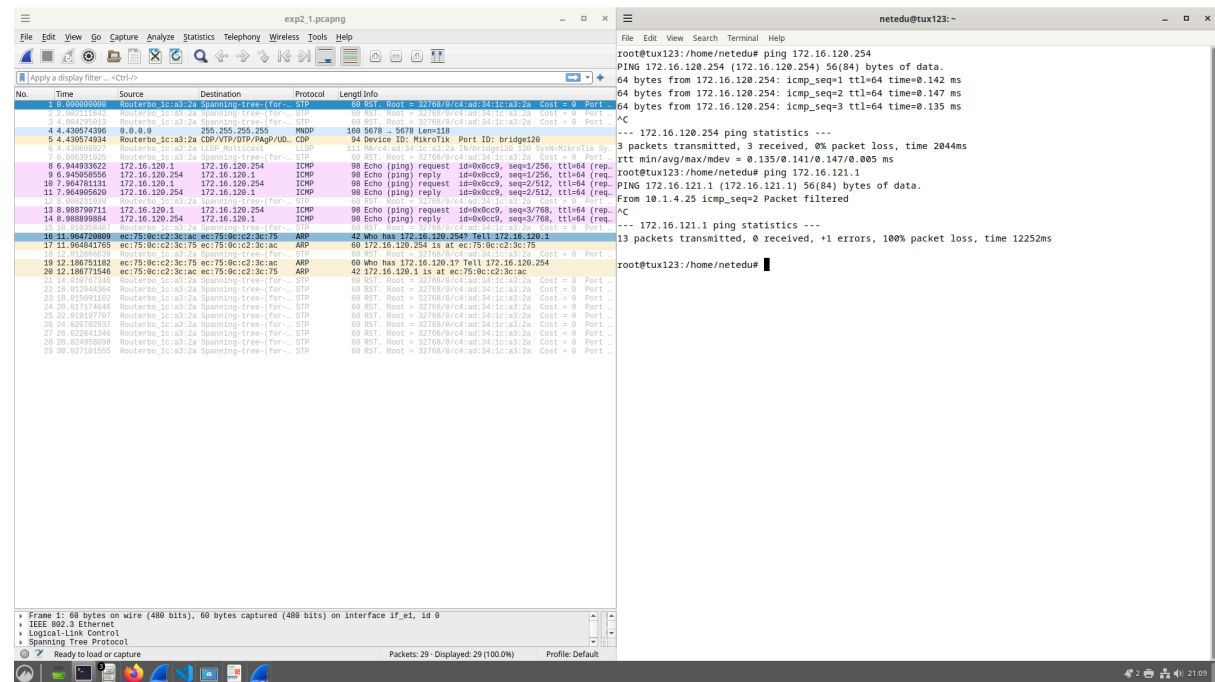
Configuring IP

- ifconfig if_e1 172.16.121.1/24 up

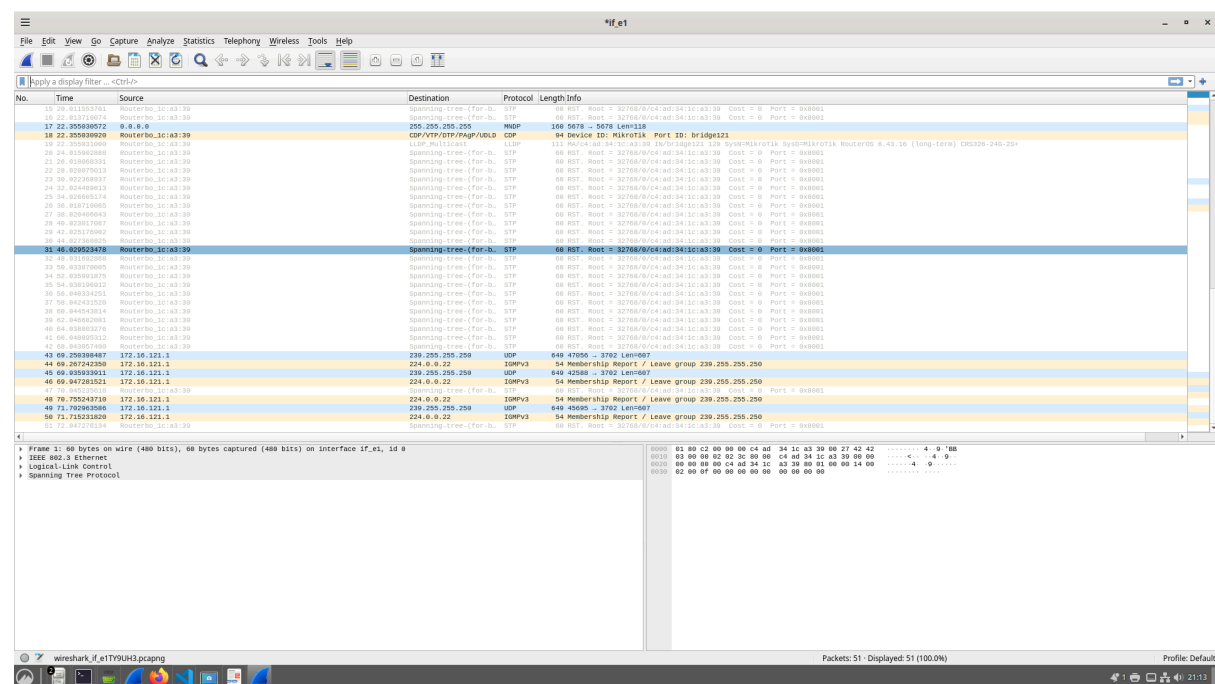
Pings Exp2

- ping 172.16.120.254 #from tux123
- ping - 172.16.121.1 #from tux 123

Ping from tux123 to tux124 and tux122



Broadcast 3 to 2



Broadcast from 2

Wireshark capture of a broadcast from 172.16.121.1. The packet list shows a series of ICMP Echo (ping) requests from 172.16.121.1 to 172.16.121.255. The packet details show the ICMP Echo (ping) request structure. The packet bytes show the raw data of the ICMP Echo request.

Frame 1: 66 bytes on wire (480 bits), 66 bytes captured (480 bits) on interface if_e1, id 0

Logical Link Control

Spanning Tree Protocol

Packets: 70 Displayed: 70 (100.0%)

Broadcast from 3

Wireshark capture of a broadcast from 172.16.120.254. The packet list shows a series of ICMP Echo (ping) requests from 172.16.120.254 to 172.16.120.255. The packet details show the ICMP Echo (ping) request structure. The packet bytes show the raw data of the ICMP Echo request.

Frame 1: 66 bytes on wire (480 bits), 66 bytes captured (480 bits) on interface if_e1, id 0

Logical Link Control

Spanning Tree Protocol

Packets: 86 Displayed: 86 (100.0%)

Broadcast 2 to 3

FileEditViewGoCaptureAnalyzeStatisticsTelephonyWirelessToolsHelp

Apply a display filter...Ctrl+F

172.16.120.254

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

239.255.255.250

224.0.0.22

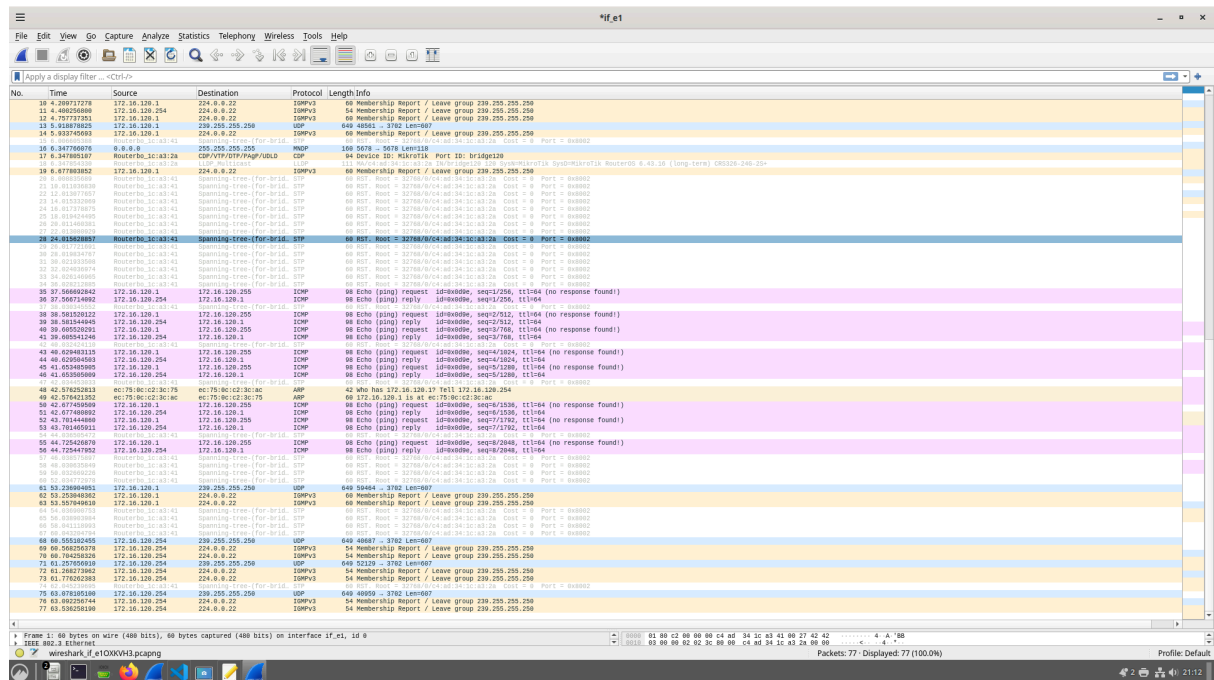
239.255.255.250

224.0.0.22

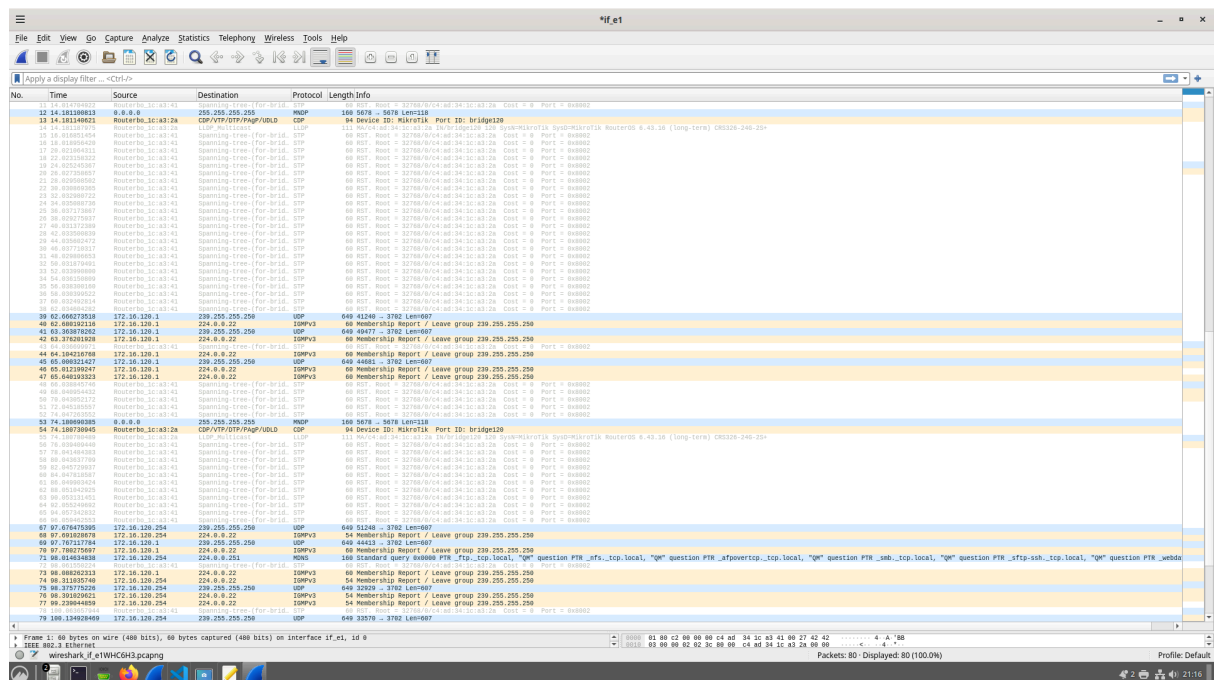
239.255.255.250

224.

Broadcast 3 to 4



Broadcast 2 to 4



Exp 3

exp3_b1

- /interface bridge port remove [find interface=ether8]
- /interface bridge port add bridge=bridge121 interface=ether8

exp3_b2

- `sysctl -w net.ipv4.ip_forward=1`
- `sysctl -w net.ipv4.icmp_echo_ignore_broadcasts=0`
- `ifconfig if_e2 172.16.121.253/24 up`

exp3_b3

- `route add -net 172.16.121.0/24 gw 172.16.120.254 # from tux123`
- `route add -net 172.16.120.0/24 gw 172.16.121.253 # from tux122`

exp3_b4

- `ping 172.16.120.254 # from tux123`
- `ping 172.16.121.253 # from tux123`
- `ping 172.16.121.1 # from tux123`

Ping 172.16.120.254

The screenshot displays a Wireshark network capture on the left and a terminal window on the right. The terminal shows the configuration of a route on tux123 and the execution of a ping command to 172.16.120.254.

Wireshark Capture (Left Panel):

- Filter:** `if_e1`
- Packet List:** Shows 22 packets. Packets 1-16 are spanning tree protocol messages. Packets 17-22 are ARP requests and replies.
- Packet Details:** For packet 17, it shows the Ethernet II header (Type: ARP) and the ARP request details (Request type: 0x0001, Sender MAC: 08:00:2b:01:02:03, Target MAC: 08:00:2b:01:02:04, Target IP: 172.16.120.254).
- Packet Bytes:** Shows the raw packet data in hexadecimal and ASCII.

Terminal Window (Right Panel):

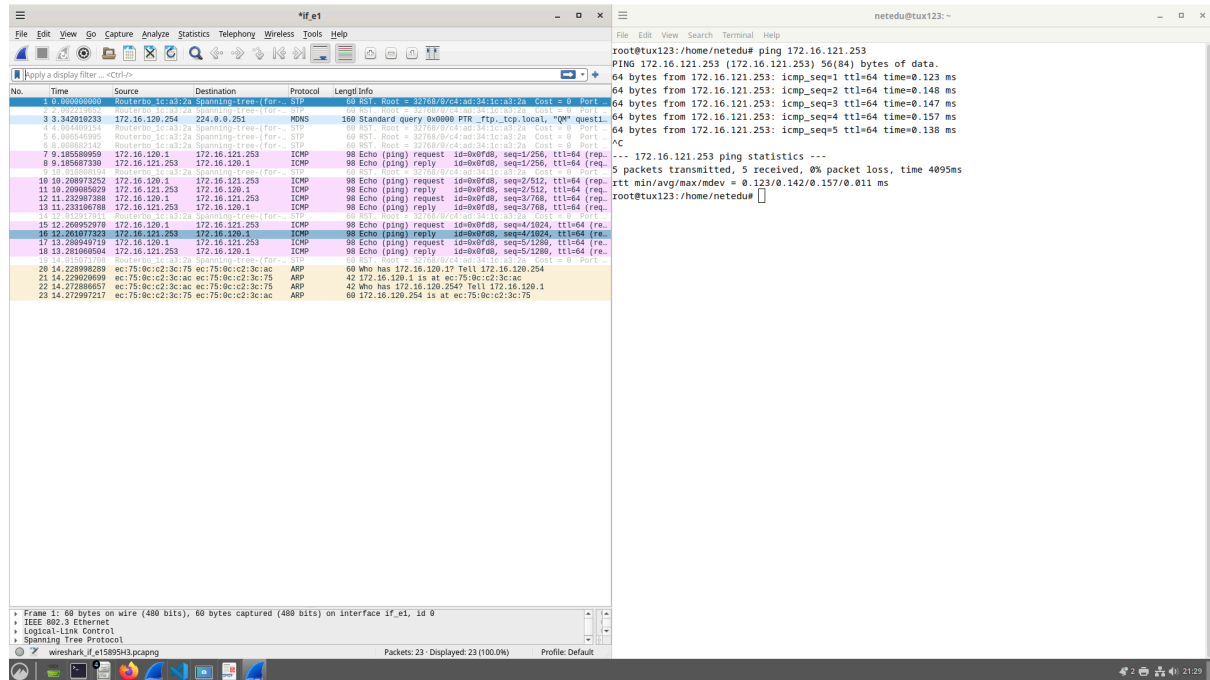
```
root@tux123:~/home/netedu# route add -net 172.16.121.0/24 gw 172.16.120.254
root@tux123:~/home/netedu# route -n

Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
0.0.0.0         10.227.20.254  0.0.0.0         UG    100    0        0 if_mng
10.227.20.0     0.0.0.0        0.0.0.0         U     100    0        0 if_mng
172.16.120.0    0.0.0.0        0.0.0.0         U     100    0        0 if_e1
172.16.121.0    172.16.120.254 255.255.255.0   UG    0       0        0 if_e1

root@tux123:~/home/netedu# ping 172.16.120.254
PING 172.16.120.254 (172.16.120.254) 56(84) bytes of data.
64 bytes from 172.16.120.254: icmp_seq=1 ttl=64 time=0.129 ms
64 bytes from 172.16.120.254: icmp_seq=2 ttl=64 time=0.141 ms
64 bytes from 172.16.120.254: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 172.16.120.254: icmp_seq=4 ttl=64 time=0.166 ms
^C
--- 172.16.120.254 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3072ms
rtt min/avg/max/ndev = 0.129/0.145/0.166/0.013 ms

root@tux123:~/home/netedu#
```


Ping 172.16.121.253



The image displays two side-by-side screenshots from a Kali Linux environment. The left window shows Wireshark capturing traffic on interface 'e1'. The packet list shows several DNS queries and responses, followed by ICMP Echo (ping) requests and replies. The packet details pane shows the structure of an ICMP Echo request. The right window shows a terminal session where the command 'ping 172.16.121.253' is executed. The output shows successful pings with 56(84) bytes of data, 0% packet loss, and a time of 4095ms.

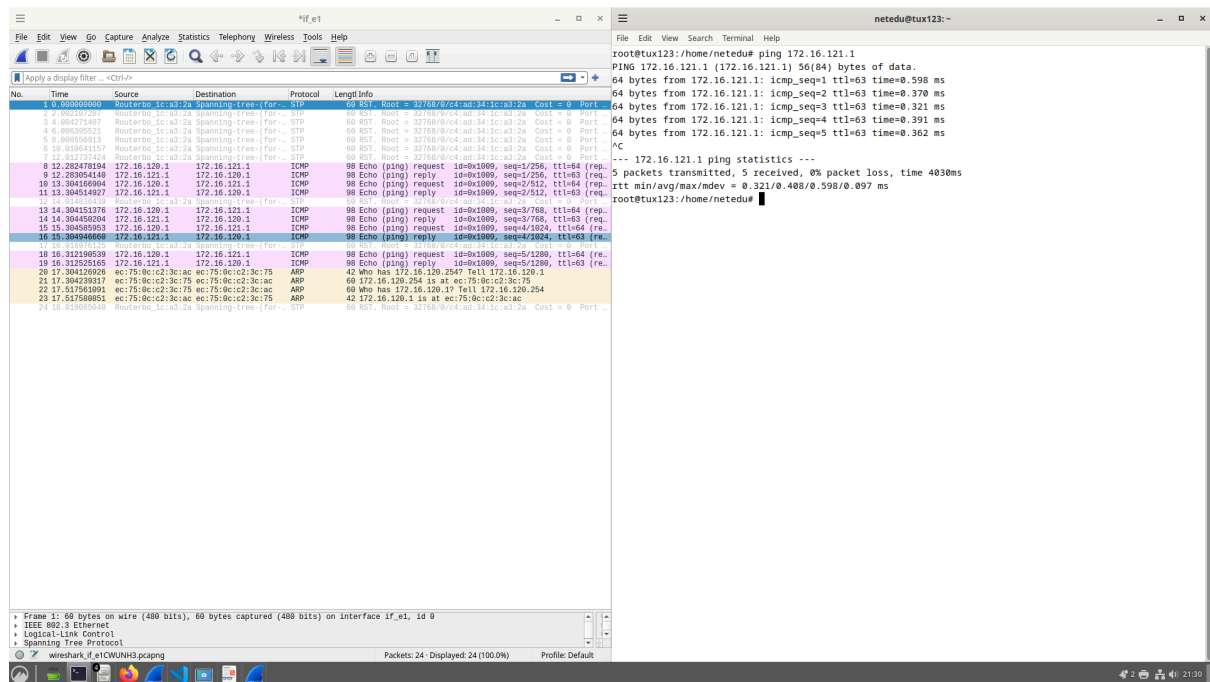
Wireshark Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
3	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
4	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
5	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
6	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
7	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
8	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
9	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
10	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
11	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
12	11.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
13	11.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
14	11.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
15	12.260000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
16	12.260000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
17	13.280000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
18	13.280000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
19	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
20	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
21	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
22	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
23	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
24	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...

Terminal Output:

```
root@tux123:~/home/netedu# ping 172.16.121.253
PING 172.16.121.253 (172.16.121.253) 56(84) bytes of data:
64 bytes from 172.16.121.253: icmp_seq=1 ttl=64 time=0.123 ms
64 bytes from 172.16.121.253: icmp_seq=2 ttl=64 time=0.140 ms
64 bytes from 172.16.121.253: icmp_seq=3 ttl=64 time=0.147 ms
64 bytes from 172.16.121.253: icmp_seq=4 ttl=64 time=0.157 ms
64 bytes from 172.16.121.253: icmp_seq=5 ttl=64 time=0.138 ms
^C
--- 172.16.121.253 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4095ms
rtt min/avg/max/ndev = 0.123/0.142/0.157/0.011 ms
root@tux123:~/home/netedu#
```

Ping 172.16.121.1



The image displays two side-by-side screenshots from a Kali Linux environment. The left window shows Wireshark capturing traffic on interface 'e1'. The packet list shows several DNS queries and responses, followed by ICMP Echo (ping) requests and replies. The packet details pane shows the structure of an ICMP Echo request. The right window shows a terminal session where the command 'ping 172.16.121.1' is executed. The output shows successful pings with 56(84) bytes of data, 0% packet loss, and a time of 4080ms.

Wireshark Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
3	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
4	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
5	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
6	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
7	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
8	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
9	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
10	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
11	0.000000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
12	11.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
13	11.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
14	11.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
15	12.260000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
16	12.260000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
17	13.280000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
18	13.280000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
19	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
20	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
21	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
22	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
23	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...
24	14.220000000	Router0_1c:a3:2a	Spanning-tree (for...) STP	60	RST	Root = 32768/0/c4:ad:34:1c:a3:2a Cost = 0 Port...

Terminal Output:

```
root@tux123:~/home/netedu# ping 172.16.121.1
PING 172.16.121.1 (172.16.121.1) 56(84) bytes of data:
64 bytes from 172.16.121.1: icmp_seq=1 ttl=63 time=0.598 ms
64 bytes from 172.16.121.1: icmp_seq=2 ttl=63 time=0.370 ms
64 bytes from 172.16.121.1: icmp_seq=3 ttl=63 time=0.321 ms
64 bytes from 172.16.121.1: icmp_seq=4 ttl=63 time=0.391 ms
64 bytes from 172.16.121.1: icmp_seq=5 ttl=63 time=0.362 ms
^C
--- 172.16.121.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4080ms
rtt min/avg/max/ndev = 0.321/0.408/0.598/0.097 ms
root@tux123:~/home/netedu#
```


Ping tux122

The image displays two side-by-side Wireshark packet capture windows. The left window, titled 'tux122', shows a successful ping from tux122 to tux124. The right window, titled 'tux124', shows a failed ping from tux122 to tux124, with a 'Destination Host Unreachable' message from tux124.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
2	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
3	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
4	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
5	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
6	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
7	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
8	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
9	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
10	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
11	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
12	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
13	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
14	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
15	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
16	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
17	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
18	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
19	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
20	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
21	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
22	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
23	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
24	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
25	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
26	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
27	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
28	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
29	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
30	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
31	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
32	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
33	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
34	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
35	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
36	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
37	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
38	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
39	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
40	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
41	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
42	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
43	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
44	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
45	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
46	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
47	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
48	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
49	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
50	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
51	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
52	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002
53	0.000000	172.16.121.1	172.16.121.254	ICMP	60	Port = 808002

Exp 4

Configuring Switch

- /interface bridge port remove [find interface=ether10]
- /interface bridge port add bridge=bridge121 interface=ether10

Configure LAN and WAN interfaces

- /ip address add address=172.16.121.254/24 interface=ether2
- /ip address add address=172.16.1.12/24 interface=ether1

Configure Routes and NAT on the Router

- /ip route add dst-address=0.0.0.0/0 gateway=172.16.1.254
- /ip route add dst-address=172.16.120.0/24 gateway=172.16.121.253
- /ip firewall nat add chain=srcnat action=masquerade out-interface=ether1

Update Routes

- sudo route del default #from tux122
- sudo route add default gw 172.16.121.254 #from tux122
- sudo route add -net 172.16.120.0/24 gw 172.16.121.253 #from tux122
- sudo route add default gw 172.16.121.254 #from tux124
- sudo route add -net 172.16.121.0/24 gw 172.16.120.254 #from tux123
- sudo route add -net 172.16.1.0/24 gw 172.16.121.254 #from tux124

Disable ICMP Redirects

- sudo sysctl net.ipv4.conf.if_e1.accept_redirects=0 #from tux122
- sudo sysctl net.ipv4.conf.all.accept_redirects=0 #from tux122

Change Gateway to RC

- sudo route del -net 172.16.120.0/24 gw 172.16.121.253 #from tux122
- sudo route add -net 172.16.120.0/24 gw 172.16.121.254 #from tux122

Restore Gateway to tux124

- sudo route del -net 172.16.120.0/24 gw 172.16.121.254 #from tux122
- sudo route add -net 172.16.120.0/24 gw 172.16.121.253 #from tux122

Enable ICMP Redirects and Test

- sudo route del -net 172.16.120.0/24 gw 172.16.121.253 #from tux122
- sudo sysctl net.ipv4.conf.if_e1.accept_redirects=1 #from tux122
- sudo sysctl net.ipv4.conf.all.accept_redirects=1 #from tux122

Disable NAT

- /ip firewall nat remove [find chain=srcnat] #from RC

Ping from 2 to 3 via Router

The image displays a Wireshark packet capture on interface 'if_e1'. The capture shows a series of ICMP Echo (ping) requests and replies. The source IP is 172.16.120.1 and the destination is 172.16.121.254. The packets are routed through 172.16.121.253. The capture includes details of the ICMP header, the IP header, and the Ethernet II header. The packet list shows 41 packets, with 100.0% displayed. The packet details pane shows the structure of the captured packets, including the Ethernet II header, the Internet Protocol Version 4 header, and the Internet Control Message Protocol header. The packet bytes pane shows the raw data of the captured packets.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.0000000	RouterBo_3c:a3:39	RouterBo_3c:a3:39	Spontaneous (For S...)	STP	60 RST: Root = 32768/0/74:40:28:00:24:12 Cost = 10 Port = 0x0001
2	0.00212140	RouterBo_3c:a3:39	RouterBo_3c:a3:39	Spontaneous (For S...)	STP	60 RST: Root = 32768/0/74:40:28:00:24:12 Cost = 10 Port = 0x0001
3	0.00427055	RouterBo_3c:a3:39	RouterBo_3c:a3:39	Spontaneous (For S...)	STP	60 RST: Root = 32768/0/74:40:28:00:24:12 Cost = 10 Port = 0x0001
4	0.00635970	RouterBo_3c:a3:39	RouterBo_3c:a3:39	Spontaneous (For S...)	STP	60 RST: Root = 32768/0/74:40:28:00:24:12 Cost = 10 Port = 0x0001
5	0.00844885	RouterBo_3c:a3:39	RouterBo_3c:a3:39	Spontaneous (For S...)	STP	60 RST: Root = 32768/0/74:40:28:00:24:12 Cost = 10 Port = 0x0001
6	0.740400173	ec:79:0c:c2:31:73	Broadcast	ARP	42	Who has 172.16.121.254? Tell 172.16.121.1
7	0.740600764	RouterBo_4b:24:12	ec:79:0c:c2:31:73	ARP	60	172.16.121.254 is at 74:40:28:00:24:12
8	0.740600920	172.16.121.1	ec:79:0c:c2:31:73	ICMP	98	Echo (ping) request 1000000, seq=1/256, ttl=64 (reply in 9)
9	0.740600978	172.16.121.1	ec:79:0c:c2:31:73	ICMP	98	Echo (ping) reply 1000000, seq=1/256, ttl=64 (request in 8)
10	0.740601040	RouterBo_3c:a3:39	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 10)
11	0.740601097	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 11)
12	0.740601154	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 12)
13	0.740601211	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 13)
14	0.740601268	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 14)
15	0.740601325	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 15)
16	0.740601382	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 16)
17	0.740601439	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 17)
18	0.740601496	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 18)
19	0.740601553	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 19)
20	0.740601610	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 20)
21	0.740601667	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 21)
22	0.740601724	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 22)
23	0.740601781	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 23)
24	0.740601838	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 24)
25	0.740601895	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 25)
26	0.740601952	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 26)
27	0.740602009	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 27)
28	0.740602066	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 28)
29	0.740602123	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 29)
30	0.740602180	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 30)
31	0.740602237	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 31)
32	0.740602294	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 32)
33	0.740602351	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 33)
34	0.740602408	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 34)
35	0.740602465	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 35)
36	0.740602522	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 36)
37	0.740602579	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 37)
38	0.740602636	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 38)
39	0.740602693	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 39)
40	0.740602750	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 40)
41	0.740602807	172.16.121.1	172.16.121.1	ICMP	64	Echo (ping) request 1000000, seq=2/512, ttl=64 (reply in 41)

[illegible]

Frame 1: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface if_e1, id 0

- Ethernet II, Src: Intel E100 (08:00:27:00:00:00), Dst: Intel E100 (08:00:27:00:00:00)
- Logical Link Control
- Spanning Tree Protocol

Wireless: if_e1:5927h3pcapng

Packets: 13 Displayed: 13 (100.0%)

Profile: Default

```
root@tux123:~# root
PING 172.16.121.1 (172.16.121.1) 56(84) bytes of data:
64 bytes from 172.16.121.1: icmp_seq=1 ttl=63 time=0.698 ms
64 bytes from 172.16.121.1: icmp_seq=2 ttl=63 time=0.416 ms
64 bytes from 172.16.121.1: icmp_seq=3 ttl=63 time=0.419 ms
64 bytes from 172.16.121.1: icmp_seq=4 ttl=63 time=0.427 ms
^C
--- 172.16.121.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3076ms
rtt min/avg/max/mdev = 0.416/0.490/0.698/0.120 ms
root@tux123:~# route -n
Kernel IP routing table
Destination Gateway Flags Metric Ref Use Iface
0.0.0.0 10.227.20.254 0.0.0.0 UG 100 0 0 if_mng
10.227.20.0 0.0.0.0 255.255.255.0 U 100 0 0 if_mng
172.16.120.0 0.0.0.0 255.255.255.0 U 0 0 0 if_e1
172.16.121.0 172.16.120.254 255.255.255.0 UG 0 0 0 if_e1
root@tux123:~# ping 172.16.120.254
PING 172.16.120.254 (172.16.120.254) 56(84) bytes of data:
64 bytes from 172.16.120.254: icmp_seq=1 ttl=64 time=0.136 ms
64 bytes from 172.16.120.254: icmp_seq=2 ttl=64 time=0.139 ms
64 bytes from 172.16.120.254: icmp_seq=3 ttl=64 time=0.151 ms
^C
--- 172.16.120.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2051ms
rtt min/avg/max/mdev = 0.136/0.142/0.151/0.006 ms
root@tux123:~#
```

ping from 3 to 4 Remote

The image shows a Wireshark packet capture and a terminal window. The Wireshark window is titled "Capturing from if e1" and displays a list of 16 packets. The first 14 packets are spanning tree protocol (STP) messages, and the last two are ARP requests. The terminal window shows the output of a ping command from root@tux123 to 172.16.121.253.

Wireshark Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
2	0.000000000	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
3	0.001958889	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
4	0.006260402	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
5	0.858888417	172.16.120.1	172.16.121.253	ICMP	96	Echo (ping) request 16=0x1150, seq=1/256, ttl=64 (req...
6	0.858812080	172.16.121.253	172.16.120.1	ICMP	96	Echo (ping) reply 16=0x1150, seq=1/256, ttl=64 (req...
7	0.869425952	172.16.120.1	172.16.121.253	ICMP	96	Echo (ping) request 16=0x1150, seq=2/512, ttl=64 (req...
8	0.869503370	172.16.121.253	172.16.120.1	ICMP	96	Echo (ping) reply 16=0x1150, seq=2/512, ttl=64 (req...
9	0.893424487	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
10	0.893424487	172.16.120.1	172.16.121.253	ICMP	96	Echo (ping) request 16=0x1150, seq=3/768, ttl=64 (req...
11	0.893551487	172.16.121.253	172.16.120.1	ICMP	96	Echo (ping) reply 16=0x1150, seq=3/768, ttl=64 (req...
12	0.918854726	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
13	11.918854726	ec:75:0c:c2:3c:75	ec:75:0c:c2:3c:ac	ARP	60	Who has 172.16.120.17 Tell 172.16.120.254
14	11.918877446	ec:75:0c:c2:3c:ac	ec:75:0c:c2:3c:75	ARP	42	172.16.120.1 is at ec:75:0c:c2:3c:ac
15	12.013359237	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
16	14.013359237	ec:75:0c:c2:3c:ac	ec:75:0c:c2:3c:75	ARP	42	Who has 172.16.120.254? Tell 172.16.120.1

Terminal Output:

```
root@tux123:/home/netedus# ping 172.16.121.253
PING 172.16.121.253 (172.16.121.253) 56(84) bytes of data.
64 bytes from 172.16.121.253: icmp_seq=1 ttl=64 time=0.147 ms
64 bytes from 172.16.121.253: icmp_seq=2 ttl=64 time=0.147 ms
64 bytes from 172.16.121.253: icmp_seq=3 ttl=64 time=0.156 ms
^C
--- 172.16.121.253 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2035ms
rtt min/avg/max/mdev = 0.147/0.150/0.156/0.004 ms
root@tux123:/home/netedus#
```

Ping from 3 to 2 via 4

The image shows a Wireshark packet capture and a terminal window. The Wireshark window is titled "Capturing from if e1" and displays a list of 24 packets. The first 22 packets are spanning tree protocol (STP) messages, and the last two are ARP requests. The terminal window shows the output of a ping command from root@tux123 to 172.16.121.1.

Wireshark Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
2	0.000000000	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
3	0.001958889	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
4	0.334555668	0.0.0.0	255.255.255.255	NDP	160	5678 = 5678 Len=118
5	4.334555668	Routerbo_ic-a3:2a	CDP/VDP/OTDP/PagP/UD...	CDP	94	Device ID: Mikrotik Port ID: bridge120
6	4.334555668	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	11	0x1c:ad:34-1c:a3:2a 0x1c:ad:34-1c:a3:2a 0x1c:ad:34-1c:a3:2a
7	5.993655413	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
8	7.993655413	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
9	9.999750294	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
10	12.999544503	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
11	14.903541628	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
12	16.903541628	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
13	17.952610851	172.16.120.1	172.16.121.1	ICMP	96	Echo (ping) request 16=0x1184, seq=1/256, ttl=64 (req...
14	17.953094182	172.16.121.1	172.16.120.1	ICMP	96	Echo (ping) reply 16=0x1184, seq=1/256, ttl=63 (req...
15	18.000000000	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
16	18.062358889	172.16.120.1	172.16.121.1	ICMP	96	Echo (ping) request 16=0x1184, seq=2/512, ttl=64 (req...
17	18.062358889	172.16.121.1	172.16.120.1	ICMP	96	Echo (ping) reply 16=0x1184, seq=2/512, ttl=63 (req...
18	18.886334114	172.16.120.1	172.16.121.1	ICMP	96	Echo (ping) request 16=0x1184, seq=3/768, ttl=64 (req...
19	19.886691203	172.16.121.1	172.16.120.1	ICMP	96	Echo (ping) reply 16=0x1184, seq=3/768, ttl=63 (req...
20	20.916454007	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
21	22.916454007	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...
22	22.979226109	ec:75:0c:c2:3c:75	ec:75:0c:c2:3c:ac	ARP	60	Who has 172.16.120.17 Tell 172.16.120.254
23	22.979247458	ec:75:0c:c2:3c:ac	ec:75:0c:c2:3c:75	ARP	42	172.16.120.1 is at ec:75:0c:c2:3c:ac
24	24.013359237	Routerbo_ic-a3:2a	Spanning-tree (for...	STP	60	RST Root = 32768/0/c4-ad:34-1c:a3:2a Cost = 0 Port...

Terminal Output:

```
root@tux123:/home/netedus# ping 172.16.121.1
PING 172.16.121.1 (172.16.121.1) 56(84) bytes of data.
64 bytes from 172.16.121.1: icmp_seq=1 ttl=63 time=0.444 ms
64 bytes from 172.16.121.1: icmp_seq=2 ttl=63 time=0.370 ms
64 bytes from 172.16.121.1: icmp_seq=3 ttl=63 time=0.386 ms
^C
--- 172.16.121.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2034ms
rtt min/avg/max/mdev = 0.370/0.400/0.444/0.031 ms
root@tux123:/home/netedus#
```

Ping from 3 to Router

The image shows a Wireshark packet capture and a terminal window. The Wireshark window is titled "Capturing on if_e1" and shows a list of packets. The first packet is an ICMP Echo (ping) request from 172.16.121.254 to 172.16.121.254. The terminal window shows the command "root@tux123:~# ping 172.16.121.254" and the output of the ping command, which shows 4 successful pings with 0% packet loss.

Wireshark Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
2	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
3	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
4	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
5	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
6	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
7	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
8	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
9	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
10	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
11	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
12	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
13	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
14	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
15	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
16	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
17	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
18	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
19	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
20	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
21	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
22	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
23	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
24	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...

Terminal Output:

```
root@tux123:~# ping 172.16.121.254
PING 172.16.121.254 (172.16.121.254) 56(84) bytes of data.
64 bytes from 172.16.121.254: icmp_seq=1 ttl=63 time=0.532 ms
64 bytes from 172.16.121.254: icmp_seq=2 ttl=63 time=0.324 ms
64 bytes from 172.16.121.254: icmp_seq=3 ttl=63 time=0.342 ms
64 bytes from 172.16.121.254: icmp_seq=4 ttl=63 time=0.351 ms
^C
--- 172.16.121.254 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3073ms
rtt min/avg/max/mdev = 0.324/0.387/0.532/0.084 ms
root@tux123:~#
```

Ping from 3 to Server step5

The image shows a Wireshark packet capture and a terminal window. The Wireshark window is titled "if_e1" and shows a list of packets. The first packet is an ICMP Echo (ping) request from 172.16.121.254 to 172.16.1.10. The terminal window shows the command "root@tux123:~# ping 172.16.1.10" and the output of the ping command, which shows 4 successful pings with 0% packet loss.

Wireshark Packet List:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
2	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
3	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
4	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
5	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
6	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
7	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
8	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
9	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
10	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
11	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
12	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
13	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
14	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
15	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
16	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
17	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
18	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
19	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
20	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
21	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
22	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
23	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...
24	0.000000000	Router00_1c:a3:2a	Spanning-tree (for...	STP	60	RST: Root = 32768/0:c4:8d:34:1c:a3:2a Cost = 0 Port...

Terminal Output:

```
root@tux123:~# ping 172.16.1.10
PING 172.16.1.10 (172.16.1.10) 56(84) bytes of data.
64 bytes from 172.16.1.10: icmp_seq=1 ttl=62 time=0.624 ms
64 bytes from 172.16.1.10: icmp_seq=2 ttl=62 time=0.498 ms
64 bytes from 172.16.1.10: icmp_seq=3 ttl=62 time=0.488 ms
64 bytes from 172.16.1.10: icmp_seq=4 ttl=62 time=0.464 ms
^C
--- 172.16.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3052ms
rtt min/avg/max/mdev = 0.408/0.498/0.624/0.079 ms
root@tux123:~#
```

Ping from 3 to server step7 fails

The left screenshot shows a Wireshark capture on interface 'if_1'. The packet list shows several ICMP Echo (ping) requests from 172.16.1.10 to 172.16.1.10, all of which are dropped. The packet details pane shows the ICMP Echo request structure. The packet bytes pane shows the raw data of the ICMP Echo request.

The right screenshot shows a terminal window on a netedutux123 machine. The user runs the command 'ping 172.16.1.10' and the output shows 'PING 172.16.1.10 (172.16.1.10) 56(84) bytes of data.' followed by '--- 172.16.1.10 ping statistics ---' and '5 packets transmitted, 0 received, 100% packet loss, time 4075ms'.

Exp 5

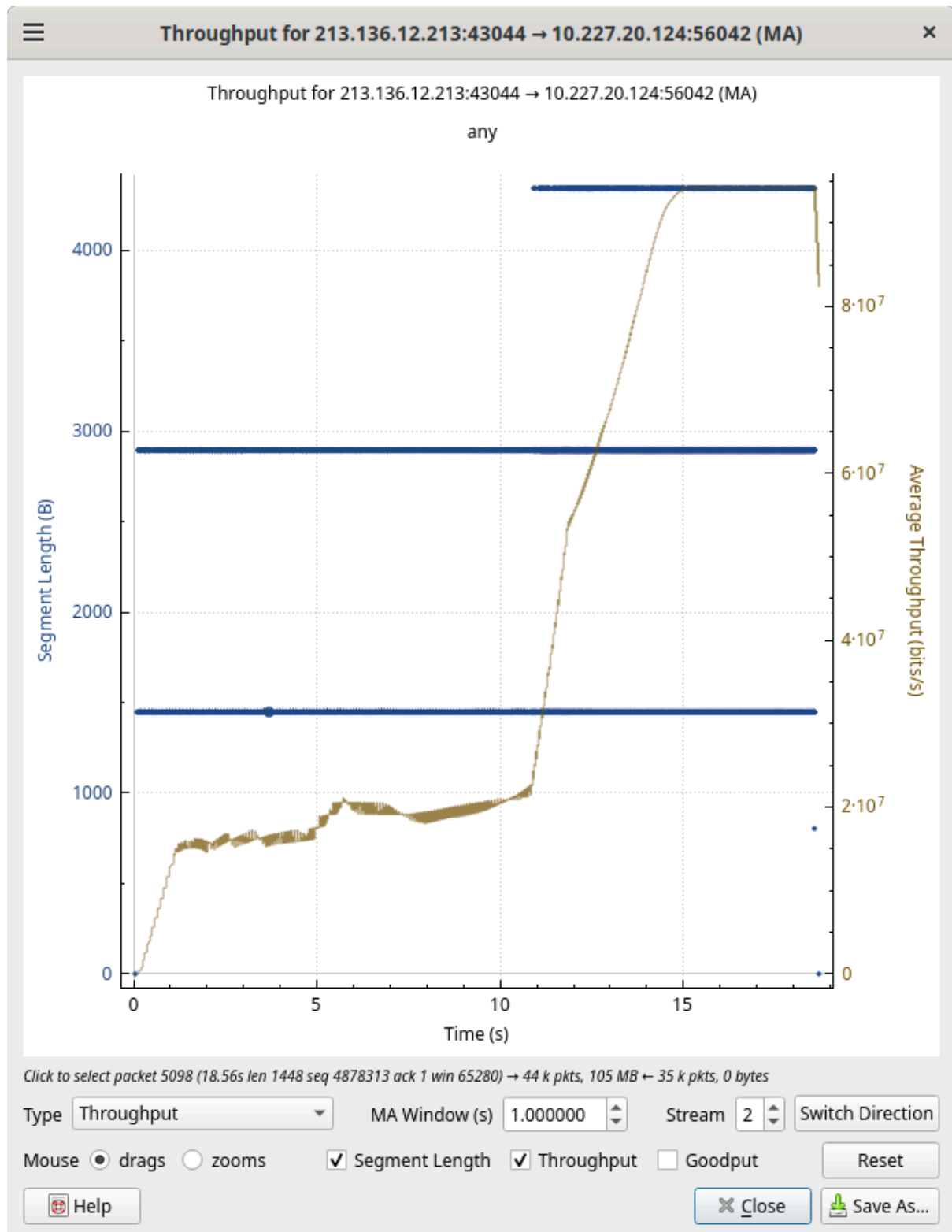
Ping Google.com

The left screenshot shows a Wireshark capture on interface 'if_mng'. The packet list shows several ICMP Echo (ping) requests from 172.16.1.10 to 172.16.1.10, all of which are dropped. The packet details pane shows the ICMP Echo request structure. The packet bytes pane shows the raw data of the ICMP Echo request.

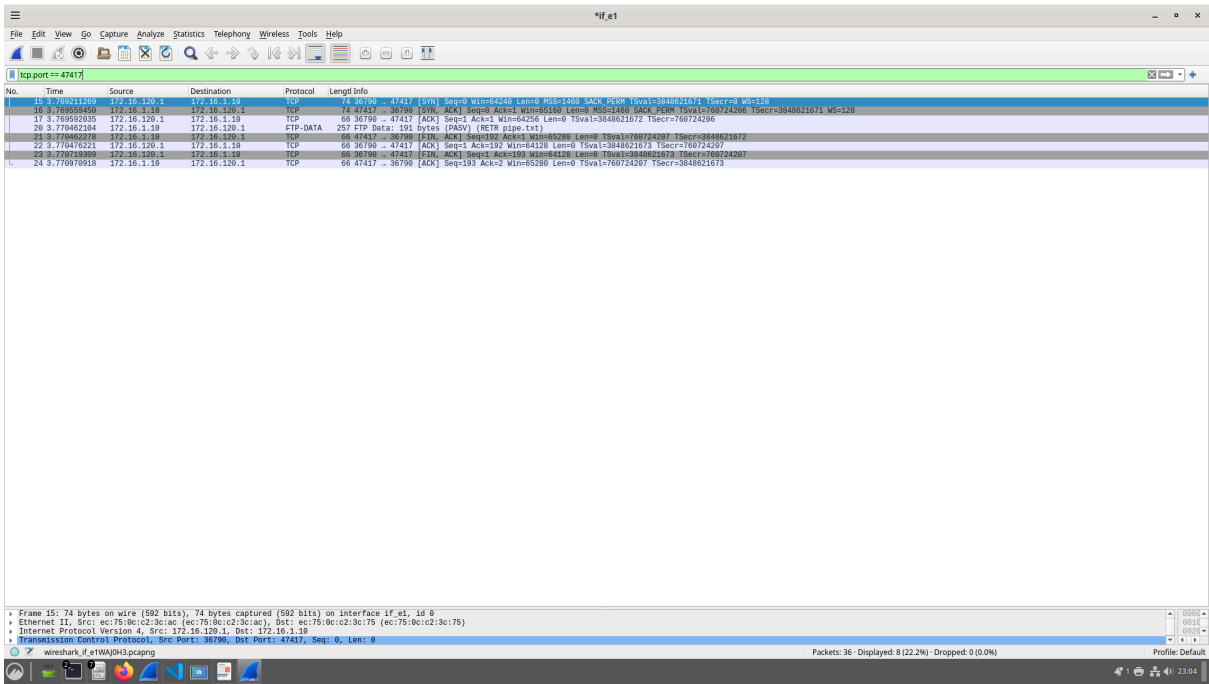
The right screenshot shows a terminal window on a netedutux123 machine. The user runs the command 'ping google.com' and the output shows 'PING google.com (142.250.200.142) 56(84) bytes of data.' followed by '64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=1 ttl=112 time=16.8 ms' and '64 bytes from mad41s14-in-f14.1e100.net (142.250.200.142): icmp_seq=2 ttl=112 time=15.9 ms'.

Exp 6

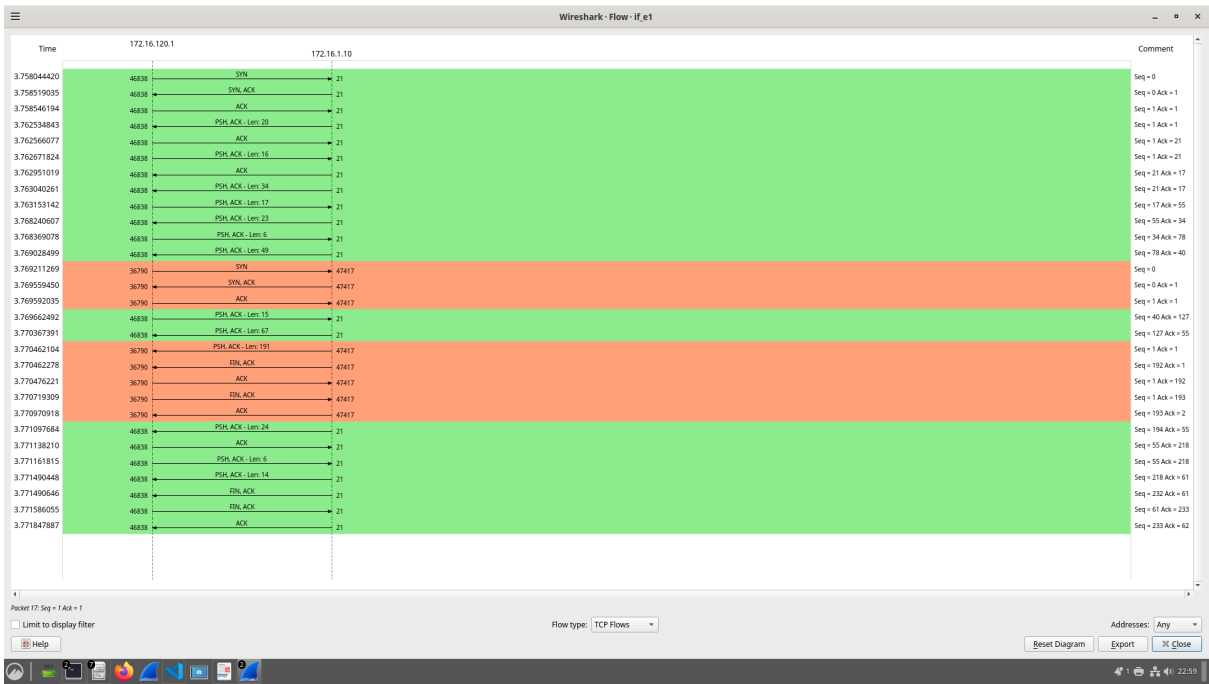
Sametime_final_tux4



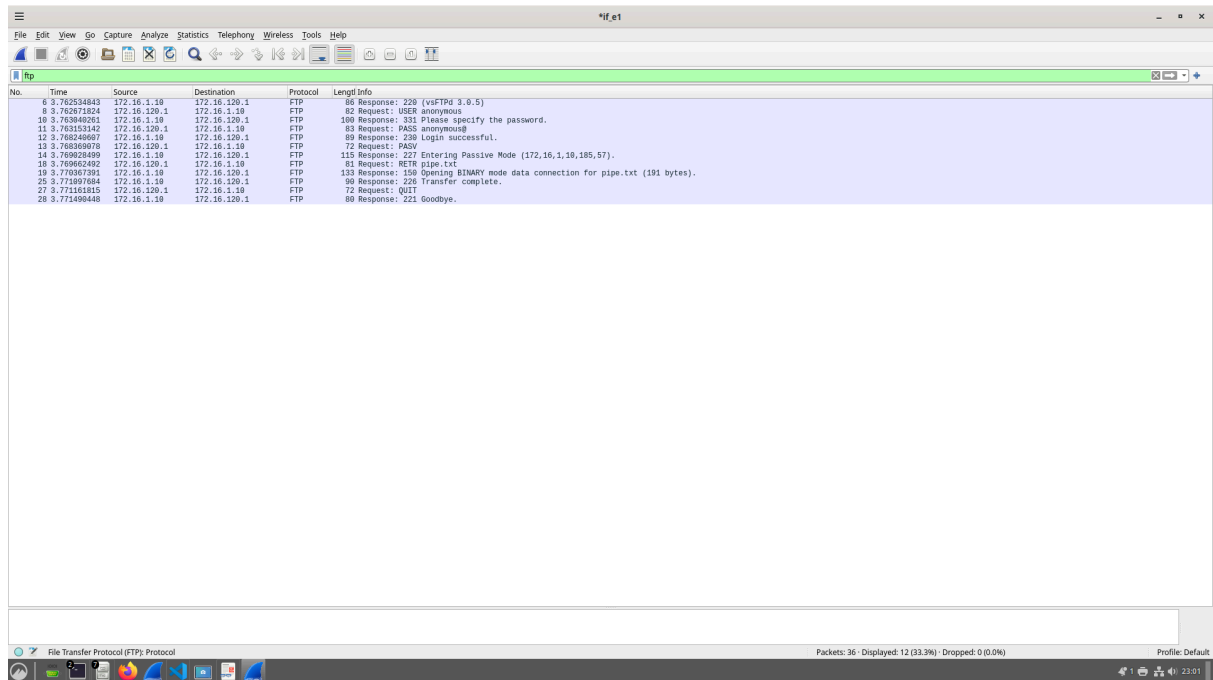
step4 tcp arq



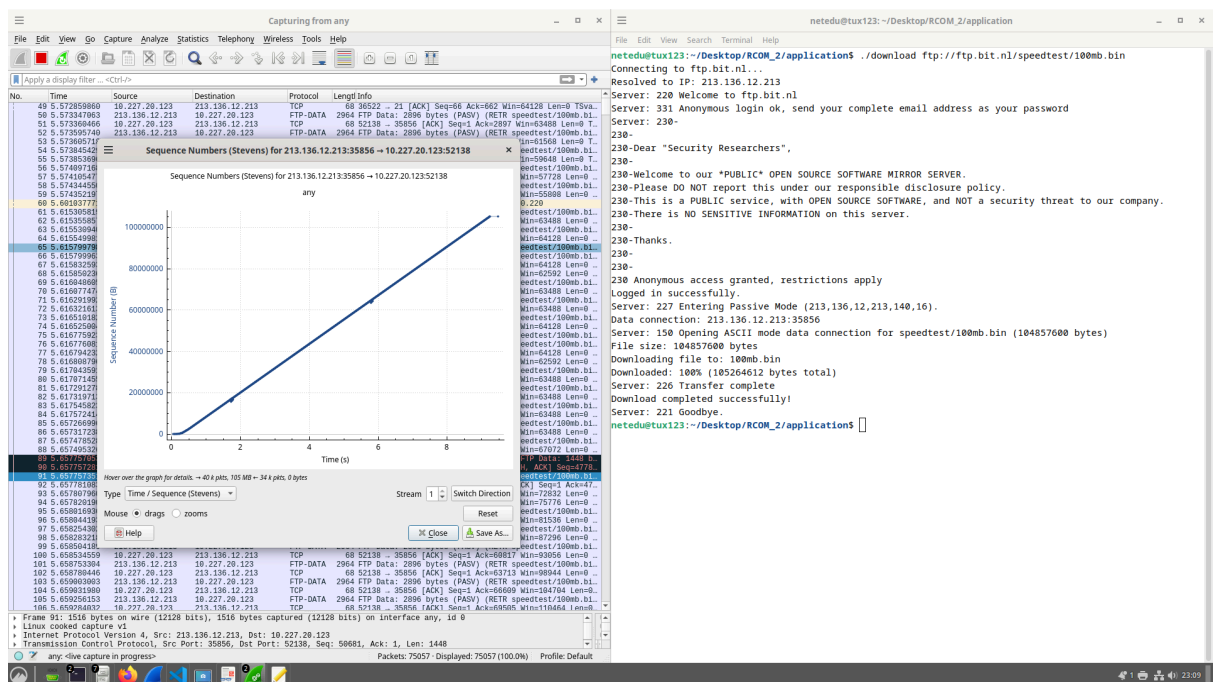
step4 tcp



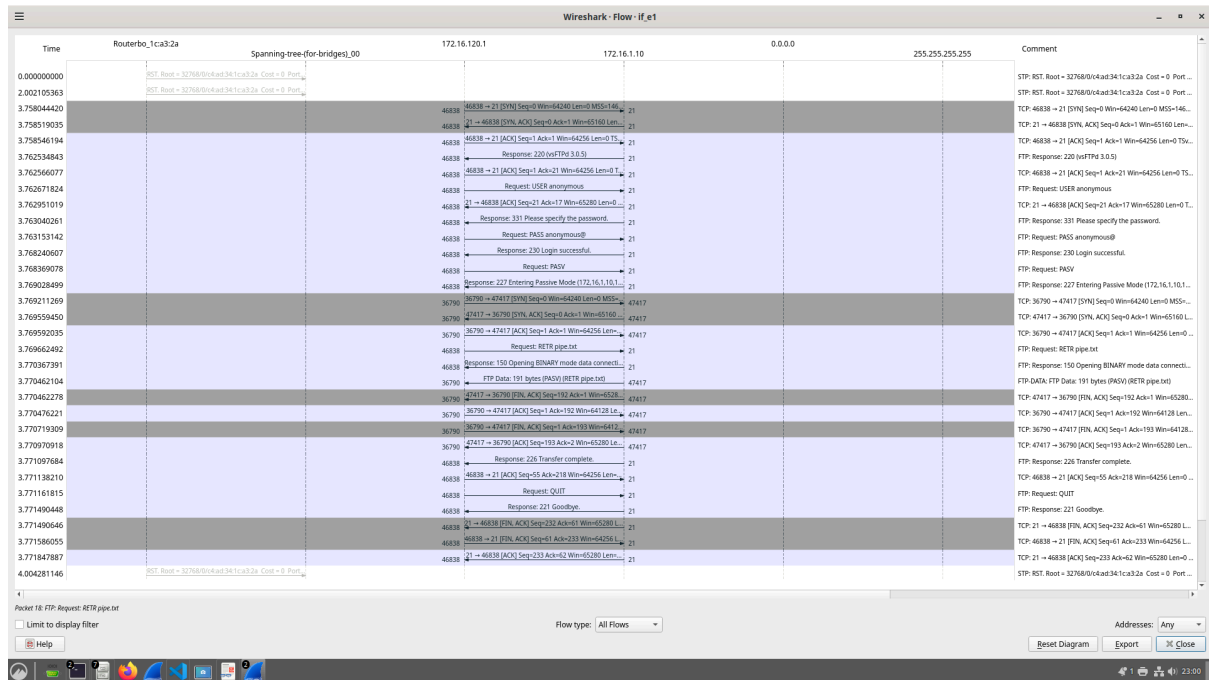
step 4 ftp filter



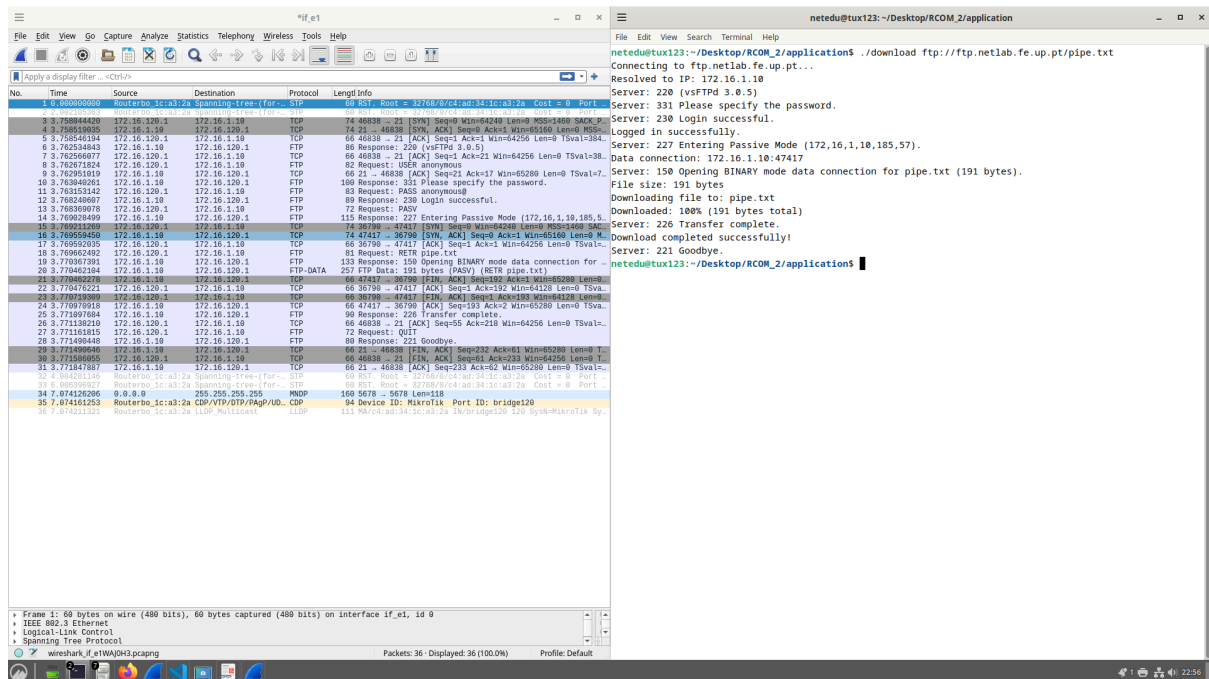
step 4 congestion control



step 4 all flows



step 2



sametime final tux3

